

Отчёт по лабораторной работе №4

Имитационное моделирование

Ибрахим Мохсейн Алькамаль

Содержание

1	Цель работы	7
2	Выполнение лабораторной работы	8
2.1	Подготовка рабочего проекта	8
2.2	Базовый эксперимент модели SIR	9
2.3	Преобразование базового эксперимента в литературный стиль	9
2.4	Выполнение базового эксперимента в Jupyter Notebook	10
2.5	Сканирование коэффициента заразности	10
2.6	Преобразование сканирования коэффициента заразности в литературный стиль	11
2.7	Анализ коэффициента заразности в Jupyter Notebook	12
2.8	Исследование влияния миграции	12
2.9	Преобразование исследования миграции в литературный стиль	13
2.10	Анализ влияния миграции в Jupyter Notebook	14
2.11	Оптимизация параметров модели	14
2.12	Преобразование оптимизации в литературный стиль	15
2.13	Выполнение оптимизации в Jupyter Notebook	16
2.14	Комплексная визуализация результатов	17
2.15	Преобразование визуализации в литературный стиль	17
2.16	Анализ результатов в Jupyter Notebook	17
3	Базовый эксперимент агентной модели SIR	19
3.1	Подключение библиотек	19
3.2	Параметры эксперимента	20
3.3	Инициализация модели	21
3.4	Подготовка данных	22
3.5	Запуск моделирования	22
3.6	Формирование таблиц результатов	23
3.7	Визуализация динамики эпидемии	25
3.8	Сохранение результатов	27
3.9	Вывод	28
4	Исследование влияния коэффициента заразности β	29
4.1	Подключение библиотек	29
4.2	Функция выполнения одного эксперимента	30
4.3	Диапазон исследуемых параметров	33

4.4	Формирование набора параметров	34
4.5	Выполнение серии экспериментов	35
4.6	Формирование таблицы результатов	37
4.7	Сохранение результатов	39
4.8	Усреднение результатов	40
4.9	Визуализация результатов	41
4.10	Результаты эксперимента	44
4.11	Вывод	45
5	Исследование влияния интенсивности миграции	46
5.1	Подключение библиотек	46
5.2	Формирование матрицы миграции	47
5.3	Определение времени достижения пика эпидемии	48
5.4	Диапазон интенсивности миграции	51
5.5	Формирование набора параметров	51
5.6	Выполнение серии экспериментов	53
5.7	Формирование таблицы результатов	54
5.8	Сохранение результатов	56
5.9	Усреднение результатов	56
5.10	Визуализация результатов	58
5.11	Результаты эксперимента	60
5.12	Вывод	60
6	Многокритериальная оптимизация параметров модели SIR	61
6.1	Подключение библиотек	61
6.2	Целевая функция многокритериальной оптимизации	62
6.3	Диапазоны оптимизируемых параметров	66
6.4	Запуск многокритериальной оптимизации	66
6.5	Получение результата оптимизации	68
6.6	Вывод оптимальных параметров	68
6.7	Сохранение результатов	69
6.8	Вывод	69
7	Комплексный анализ результатов сканирования β	71
7.1	Подключение библиотек	71
7.2	Загрузка результатов экспериментов	72
7.3	Анализ доли инфицированных	74
7.4	Анализ смертности	76
7.5	Анализ выздоровления	77
7.6	Формирование составного графика	78
7.7	Сохранение результата	80
7.8	Результат анализа	81
7.9	Вывод	81

Список иллюстраций

2.1	Инициализация рабочего проекта и установка зависимостей Julia . .	8
2.2	Проверка установленных пакетов проектного окружения	9
2.3	Выполнение базового эксперимента модели SIR и сохранение результатов	9
2.4	Генерация скрипта, документа Quarto и Jupyter Notebook для базового эксперимента SIR	10
2.5	График динамики состояний агентной модели SIR в Jupyter Notebook	10
2.6	Выполнение параметрического сканирования коэффициента заразности модели SIR	11
2.7	Генерация файлов литературной версии сканирования коэффициента заразности	11
2.8	Зависимость пиковой и конечной доли инфицированных от коэффициента заразности	12
2.9	Выполнение серии экспериментов с различной интенсивностью миграции	13
2.10	Сохранение результатов исследования влияния миграции	13
2.11	Генерация файлов литературной версии исследования влияния миграции	14
2.12	Анализ зависимости показателей эпидемии от интенсивности миграции	14
2.13	Выполнение многокритериальной оптимизации параметров SIR-модели	15
2.14	Генерация файлов литературной версии многокритериальной оптимизации	16
2.15	Запуск многокритериальной оптимизации в Jupyter Notebook	16
2.16	Выполнение скрипта комплексной визуализации результатов модели SIR	17
2.17	Генерация файлов литературной версии комплексной визуализации модели SIR	17
2.18	Комплексный анализ показателей агентной модели SIR в зависимости от коэффициента заразности	18

Список таблиц

1 Цель работы

Изучить реализацию эпидемиологической модели SIR в агентном подходе. Выполнить базовое моделирование и исследовать влияние коэффициента заразности на динамику распространения инфекции.


```
% Run the simulation model for the first time
library(J)
# Setup up scripts/run_basic_library.J
## Generating plain script file from ~/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab94/project/scripts/sir_run_basic_library.R
info: writing result to ~/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab94/project/scripts/sir_run_basic_library.J
## Creating notebook file from ~/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab94/project/scripts/sir_run_basic_library.R
Notebook output: /home/akamal/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab94/project/scripts/sir_run_basic_library/sir_run_basic_library.J
info: generating markdown page from ~/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab94/project/scripts/sir_run_basic_library.J
info: writing result to ~/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab94/project/markdown/sir_run_basic_library/sir_run_basic_library.md
Quarto: /home/akamal/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab94/project/notebooks/sir_run_basic_library/sir_run_basic_library.J
info: generating notebook from ~/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab94/project/scripts/sir_run_basic_library.J
info: writing result to ~/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab94/project/notebooks/sir_run_basic_library/sir_run_basic_library.ipynb
Notebook: /home/akamal/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab94/project/notebooks/sir_run_basic_library/sir_run_basic_library.ipynb
```

Рисунок 2.4: Генерация скрипта, документа Quarto и Jupyter Notebook для базового эксперимента SIR

2.4 Выполнение базового эксперимента в Jupyter Notebook

Сгенерированный `sir_run_basic_literary.ipynb` выполнен в Jupyter Notebook. Получен график динамики восприимчивых, инфицированных и выздоровевших агентов, а также общей численности популяции в течение 100 дней (рис. 2.5).

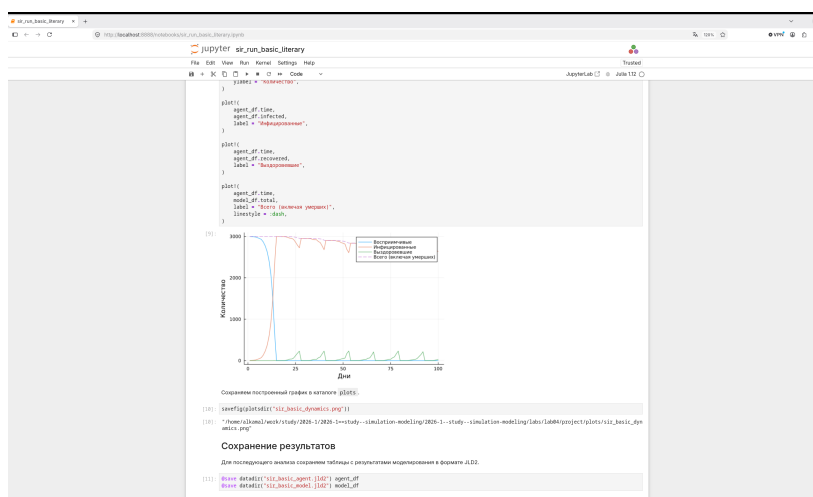


Рисунок 2.5: График динамики состояний агентной модели SIR в Jupyter Notebook

2.5 Сканирование коэффициента заразности

Для исследования влияния коэффициента заразности выполнен скрипт `sir_scan_beta.jl`. Проведены эксперименты для значений β от 0.1 до 1.0 с

тремя значениями seed: 42, 43 и 44. Результаты сохранены в `beta_scan_all.csv`, а график — в `beta_scan.png` (рис. 2.6).

```

alkamal@fedora:~/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project$ nano scripts/sir_scan_beta.jl
alkamal@fedora:~/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project$ julia --project= scripts/sir_scan_beta.jl
Запускаю эксперимент с beta = 0.1, seed = 42
Запускаю эксперимент с beta = 0.1, seed = 43
Запускаю эксперимент с beta = 0.1, seed = 44
Запускаю эксперимент с beta = 0.2, seed = 42
Запускаю эксперимент с beta = 0.2, seed = 43
Запускаю эксперимент с beta = 0.2, seed = 44
Запускаю эксперимент с beta = 0.3, seed = 42
Запускаю эксперимент с beta = 0.3, seed = 43
Запускаю эксперимент с beta = 0.3, seed = 44
Запускаю эксперимент с beta = 0.4, seed = 42
Запускаю эксперимент с beta = 0.4, seed = 43
Запускаю эксперимент с beta = 0.4, seed = 44
Запускаю эксперимент с beta = 0.5, seed = 42
Запускаю эксперимент с beta = 0.5, seed = 43
Запускаю эксперимент с beta = 0.5, seed = 44
Запускаю эксперимент с beta = 0.6, seed = 42
Запускаю эксперимент с beta = 0.6, seed = 43
Запускаю эксперимент с beta = 0.6, seed = 44
Запускаю эксперимент с beta = 0.7, seed = 42
Запускаю эксперимент с beta = 0.7, seed = 43
Запускаю эксперимент с beta = 0.7, seed = 44
Запускаю эксперимент с beta = 0.8, seed = 42
Запускаю эксперимент с beta = 0.8, seed = 43
Запускаю эксперимент с beta = 0.8, seed = 44
Запускаю эксперимент с beta = 0.9, seed = 42
Запускаю эксперимент с beta = 0.9, seed = 43
Запускаю эксперимент с beta = 0.9, seed = 44
Запускаю эксперимент с beta = 1.0, seed = 42
Запускаю эксперимент с beta = 1.0, seed = 43
Запускаю эксперимент с beta = 1.0, seed = 44
Результаты сохранены в data/beta_scan_all.csv и plots/beta_scan.png
Could not create decoration from factory! Running with no decorations.
alkamal@fedora:~/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project$ ls plots
beta_scan.png  sir_basics_dynamics.png
alkamal@fedora:~/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project$ ls data
beta_scan_all.csv  exp_raw.xlsx  sir_basics_agent.jl02  sir_basics_model.jl02
alkamal@fedora:~/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project$

```

Рисунок 2.6: Выполнение параметрического сканирования коэффициента заразности модели SIR

2.6 Преобразование сканирования коэффициента заразности в литературный стиль

Для документирования параметрического эксперимента подготовлен файл `sir_scan_beta_literary.jl`. После выполнения `tangle.jl` успешно сформированы чистый Julia-скрипт, документ Quarto и Jupyter Notebook (рис. 2.7).

```

alkamal@fedora:~/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project$ nano scripts/sir_scan_beta_literary.jl
alkamal@fedora:~/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project$ julia --project= scripts/tangle.jl scripts/sir_scan_beta_literary.jl
[Info] generating plain script file from "/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project/scripts/sir_scan_beta_literary.jl"
[Info] writing result to "/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project/scripts/sir_scan_beta_literary/sir_scan_beta_literary.jl"
[Info] Чистый скрипт: /home/alkamal/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project/scripts/sir_scan_beta_literary/sir_scan_beta_literary.jl
[Info] generating markdown page from "/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project/scripts/sir_scan_beta_literary.jl"
[Info] writing result to "/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project/markdown/sir_scan_beta_literary/sir_scan_beta_literary.qmd"
[Info] Quarto: /home/alkamal/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project/markdown/sir_scan_beta_literary/sir_scan_beta_literary.qmd
[Info] generating notebook from "/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project/scripts/sir_scan_beta_literary.jl"
[Info] writing result to "/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project/notebooks/sir_scan_beta_literary/sir_scan_beta_literary.ipynb"
[Info] Notebook: /home/alkamal/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project/notebooks/sir_scan_beta_literary/sir_scan_beta_literary.ipynb
[Done] See below output.
alkamal@fedora:~/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project$

```

Рисунок 2.7: Генерация файлов литературной версии сканирования коэффициента заразности

2.7 Анализ коэффициента заразности в Jupyter Notebook

В Jupyter Notebook построены графики зависимости эпидемических показателей от коэффициента заразности β . На графиках представлены пиковая и конечная доли инфицированных для исследуемого диапазона значений β (рис. 2.8).

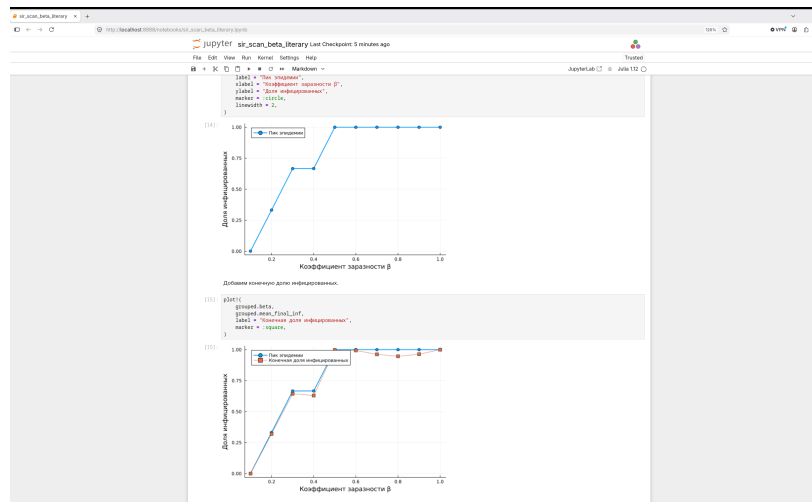


Рисунок 2.8: Зависимость пиковой и конечной доли инфицированных от коэффициента заразности

2.8 Исследование влияния миграции

Для исследования влияния миграции выполнен скрипт `sir_migration_effect.jl`. Проведена серия экспериментов с различными значениями параметра `migration_intensity` и значениями `seed` 42, 43 и 44 (рис. 2.9).

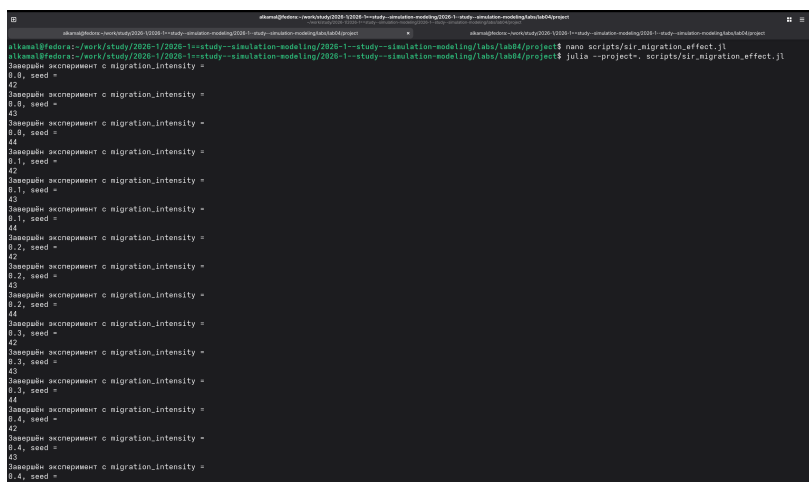


Рисунок 2.9: Выполнение серии экспериментов с различной интенсивностью миграции

После завершения экспериментов результаты сохранены в файле `data/migration_scan_all.csv`, а график исследования — в файле `plots/migration_effect.png`. Наличие созданных файлов проверено в каталогах `data` и `plots` (рис. 2.10).

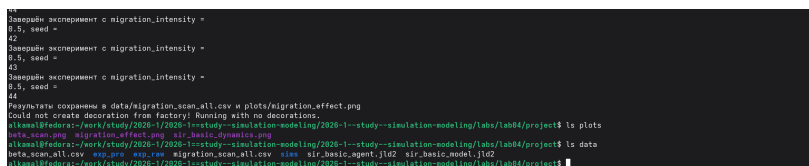


Рисунок 2.10: Сохранение результатов исследования влияния миграции

2.9 Преобразование исследования миграции в литературный стиль

Для документирования эксперимента подготовлен файл `sir_migration_effect_literary.jl`. С помощью скрипта `tangle.jl` автоматически сформированы чистый Julia-скрипт, документ Quarto и Jupyter Notebook (рис. 2.11).

borg_moea, а критериями оптимизации являлись пиковая заболеваемость и доля умерших.

В результате работы алгоритма получен набор параметров, обеспечивающий малые значения обоих критериев. Найденное значение параметра передачи инфекции составило $\beta_{und} \approx 0.2661$, время выявления инфекции — 4 дня, а смертность — приблизительно 0.0704. При этих параметрах пиковая заболеваемость составила около 0.00173, а доля умерших — около $6.67 \cdot 10^{-5}$. Результат оптимизации сохранён в файле `data/optimization_result.jld2` (рис. 2.13).

[illegible]

Рисунок 2.13: Выполнение многокритериальной оптимизации параметров SIR-модели

2.12 Преобразование оптимизации в литературный стиль

Для документирования процедуры оптимизации подготовлен файл `sir_optimize_parameters_literary.jl`. С помощью скрипта `tangle.jl` выполнено автоматическое преобразование литературного исходного файла. В результате сформированы чистый Julia-скрипт, документ Quarto и Jupyter Notebook (рис. 2.14).

Рисунок 2.14: Генерация файлов литературной версии многокритериальной оптимизации

Сгенерированный файл `sir_optimize_parameters_literary.ipynb` выполнен в Jupyter Notebook. В ноутбуке задана схема `ParetoFitnessScheme(2)`, поскольку одновременно минимизируются два критерия. Для поиска используется алгоритм `borg_tmoa`, а максимальное время работы оптимизатора установлено равным 120 секундам.

[illegible]

16

2.14 Комплексная визуализация результатов

Для объединённого анализа результатов выполнен скрипт `sir_visualize_dynamics.jl`. После запуска сформирован график `comprehensive_analysis.png` в каталоге `plots` (рис. 2.16).

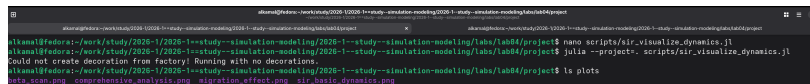


Рисунок 2.16: Выполнение скрипта комплексной визуализации результатов модели SIR

2.15 Преобразование визуализации в литературный стиль

Для документирования визуализации подготовлен файл `sir_visualize_dynamics_literary.jl`. С помощью `tangle.jl` автоматически сформированы чистый Julia-скрипт, документ Quarto и Jupyter Notebook (рис. 2.17).

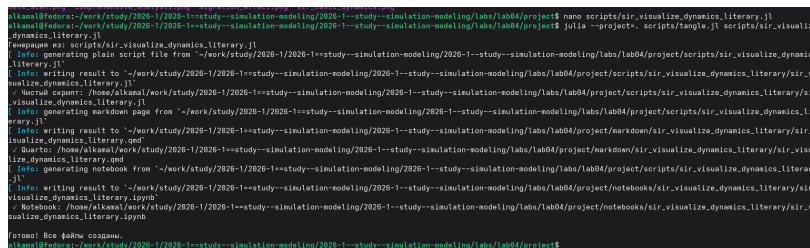


Рисунок 2.17: Генерация файлов литературной версии комплексной визуализации модели SIR

2.16 Анализ результатов в Jupyter Notebook

В Jupyter Notebook построен составной график зависимости основных показателей модели от коэффициента β . На нём представлены пиковая и конечная доли инфицированных, число умерших и доля выздоровевших (рис. 2.18).

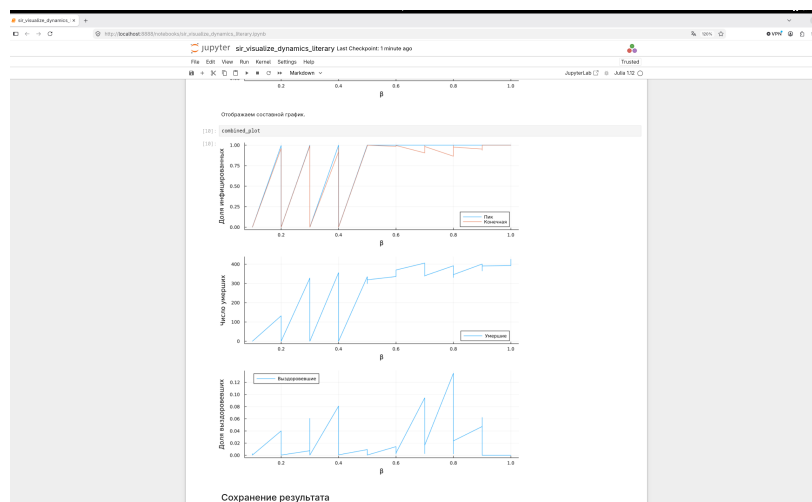


Рисунок 2.18: Комплексный анализ показателей агентной модели SIR в зависимости от коэффициента заразности

3 Базовый эксперимент агентной модели SIR

В данном эксперименте выполняется базовый запуск агентной эпидемиологической модели SIR. Популяция распределена между тремя локациями. Каждый агент может находиться в одном из трёх состояний:

- S (Susceptible) — восприимчивый;
- I (Infected) — инфицированный;
- R (Recovered) — выздоровевший.

В ходе моделирования отслеживается изменение численности восприимчивых, инфицированных и выздоровевших агентов, а также общей численности популяции.

3.1 Подключение библиотек

Активируем окружение проекта DrWatson и подключаем библиотеки, необходимые для работы с агентной моделью, табличными данными, визуализацией и сохранением результатов.

```
using DrWatson
@quickactivate "project"
```

```
using Agents
using DataFrames
using Plots
using JLD2
```

Подключаем реализацию агентной модели SIR.

```
include(scrdir("sir_model.jl"))
```

total_count (generic function with 1 method)

3.2 Параметры эксперимента

Параметры модели объединяем в словарь params.

Используются следующие значения:

- N_s — численность населения в каждой из трёх локаций;
- β_{und} — интенсивность передачи инфекции невыявленными инфицированными;
- β_{det} — интенсивность передачи инфекции выявленными инфицированными;
- infection_period — продолжительность заболевания;
- detection_time — время до выявления инфекции;
- death_rate — вероятность смерти;
- reinfection_probability — вероятность повторного заражения;
- I_s — начальное количество инфицированных в каждой локации;
- seed — зерно генератора случайных чисел;
- n_steps — продолжительность моделирования.

```
params = Dict(
    :Ns => [1000, 1000, 1000],
    :β_und => [0.5, 0.5, 0.5],
```

```

:β_det ⇒ [0.05, 0.05, 0.05],
:infection_period ⇒ 14,
:detection_time ⇒ 7,
:death_rate ⇒ 0.02,
:reinfection_probability ⇒ 0.1,
:Is ⇒ [0, 0, 1],
:seed ⇒ 42,
:n_steps ⇒ 100,
)

```

Dict{Symbol, Any} with 10 entries:

```

:Is              => [0, 0, 1]
:seed           => 42
:infection_period => 14
:n_steps        => 100
:β_und          => [0.5, 0.5, 0.5]
:Ns             => [1000, 1000, 1000]
:detection_time  => 7
:reinfection_probability => 0.1
:β_det          => [0.05, 0.05, 0.05]
:death_rate      => 0.02

```

Таким образом, начальная популяция состоит из 3000 агентов, распределённых поровну между тремя локациями. В начальный момент времени один инфицированный агент находится в третьей локации.

3.3 Инициализация модели

Создаём агентную модель SIR с заданными параметрами.

```
model = initialize_sir(; params...)
```

StandardABM with 3000 agents of type Person

agents container: Dict

space: GraphSpace with 3 positions and 3 edges

scheduler: fastest

properties: C, infection_period, β_{und} , Ns, migration_rates, detection_time, reinfection_pr

3.4 Подготовка данных

Для анализа динамики модели создаём массивы, в которые после каждого шага будут записываться модельное время и численности агентов в различных состояниях.

```
times = Int[]  
S_vals = Int[]  
I_vals = Int[]  
R_vals = Int[]  
total_vals = Int[]
```

```
Int64[]
```

3.5 Запуск моделирования

Выполняем симуляцию в течение n_steps шагов. Один шаг модели соответствует одному дню.

После каждого шага определяем:

- количество восприимчивых агентов;
- количество инфицированных;

- количество выздоровевших;
- общую численность агентов.

```
for step = 1:params[:n_steps]
    Agents.step!(model, 1)

    push!(times, step)
    push!(S_vals, susceptible_count(model))
    push!(I_vals, infected_count(model))
    push!(R_vals, recovered_count(model))
    push!(total_vals, total_count(model))
end
```

3.6 Формирование таблиц результатов

Полученные значения преобразуем в объекты DataFrame. В таблице `agent_df` хранится динамика состояний агентов, а в `model_df` — изменение общей численности популяции.

```
agent_df = DataFrame(
    time = times,
    susceptible = S_vals,
    infected = I_vals,
    recovered = R_vals,
)

model_df = DataFrame(
    time = times,
    total = total_vals,
```

)

	time	total
	Int64	Int64
1	1	3000
2	2	3000
3	3	3000
4	4	3000
5	5	3000
6	6	3000
7	7	3000
8	8	3000
9	9	3000
10	10	3000
11	11	3000
12	12	3000
13	13	3000
14	14	3000
15	15	3000
16	16	3000
17	17	3000
18	18	2999
19	19	2998
20	20	2997
21	21	2994
22	22	2993
23	23	2992
24	24	2989
...	...	...

Выведем первые строки таблицы с результатами моделирования.

```
first(agent_df, 10)
```

	time	susceptible	infected	recovered
	Int64	Int64	Int64	Int64
1	1	2998	2	0
2	2	2994	6	0
3	3	2991	9	0
4	4	2985	15	0
5	5	2969	31	0
6	6	2952	48	0
7	7	2929	71	0
8	8	2870	130	0
9	9	2779	221	0
10	10	2647	353	0

3.7 Визуализация динамики эпидемии

Построим график изменения численности восприимчивых, инфицированных и выздоровевших агентов во времени. Дополнительно пунктирной линией покажем общую численность агентов модели.

```
plot(  
    agent_df.time,  
    agent_df.susceptible,  
    label = "Восприимчивые",  
    xlabel = "Дни",  
    ylabel = "Количество",  
)
```

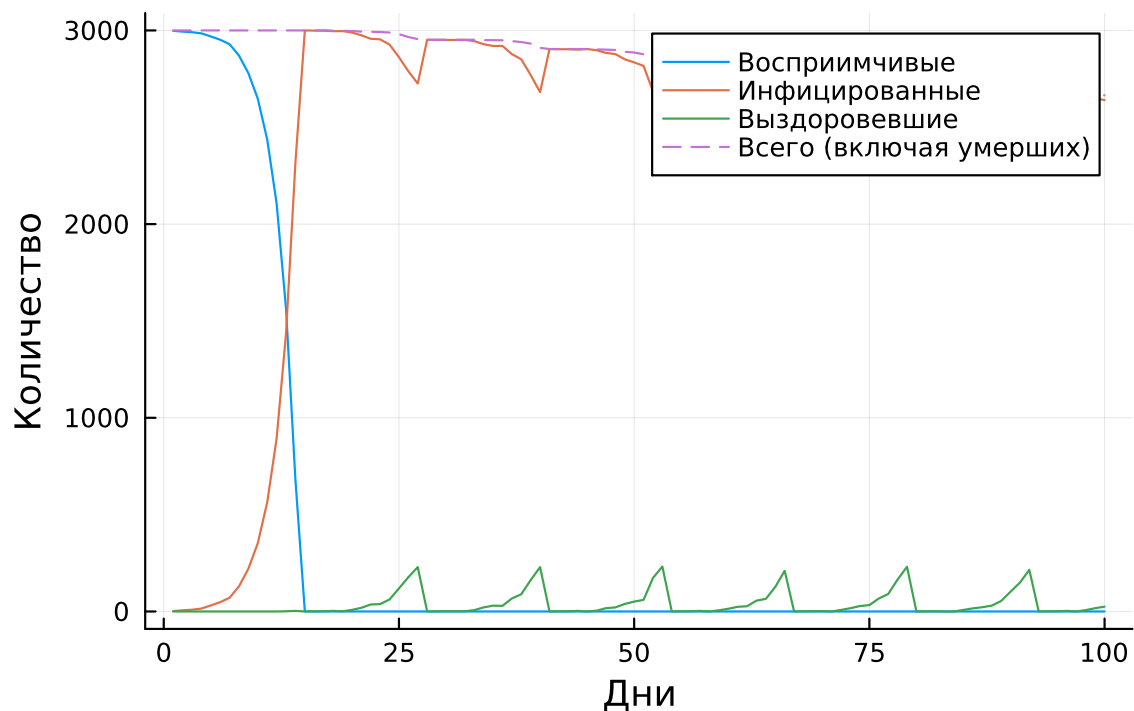
```

plot!(
    agent_df.time,
    agent_df.infected,
    label = "Инфицированные",
)

plot!(
    agent_df.time,
    agent_df.recovered,
    label = "Выздоровевшие",
)

plot!(
    agent_df.time,
    model_df.total,
    label = "Всего (включая умерших)",
    linestyle = :dash,
)

```



Сохраняем построенный график в каталоге plots.

```
savefig(plotsdir("sir_basic_dynamics.png"))
```

```
"/home/alkamal/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--  
study--simulation-modeling/labs/lab04/project/plots/sir_basic_dynamics.png"
```

3.8 Сохранение результатов

Для последующего анализа сохраняем таблицы с результатами моделирования в формате JLD2.

```
@save datadir("sir_basic_agent.jld2") agent_df  
@save datadir("sir_basic_model.jld2") model_df
```

3.9 Вывод

В результате базового эксперимента получена динамика распространения инфекции в агентной модели SIR. Моделирование позволяет проследить изменение количества восприимчивых, инфицированных и выздоровевших агентов на протяжении 100 модельных дней.

График `sir_basic_dynamics.png` позволяет визуально оценить развитие эпидемического процесса, включая рост и снижение числа инфицированных и переход агентов в состояние выздоровевших. Изменение общей численности популяции отражает влияние смертности, предусмотренной моделью.

4 Исследование влияния коэффициента заразности β

В данном эксперименте исследуется влияние коэффициента заразности β на динамику агентной эпидемиологической модели SIR.

Для каждого значения коэффициента β выполняется несколько независимых прогонов модели с различными значениями зерна генератора случайных чисел (seed). Такой подход позволяет учитывать стохастический характер агентной модели.

В ходе эксперимента определяются следующие показатели:

- максимальная доля инфицированных агентов (peak);
- конечная доля инфицированных (final_inf);
- конечная доля выздоровевших (final_rec);
- общее количество умерших (deaths).

После выполнения всех экспериментов результаты усредняются для каждого значения β и визуализируются.

4.1 Подключение библиотек

Активируем окружение проекта DrWatson и подключаем библиотеки, необходимые для агентного моделирования, обработки табличных данных, визуализации и сохранения результатов.

```
using DrWatson
@quickactivate "project"

using Agents
using DataFrames
using Plots
using CSV
using Random
using Statistics
```

Подключаем реализацию агентной модели SIR.

```
include(sourcedir("sir_model.jl"))
```

total_count (generic function with 1 method)

4.2 Функция выполнения одного эксперимента

Определим функцию `run_experiment`, которая получает словарь параметров одного эксперимента, создаёт модель SIR, выполняет моделирование и возвращает основные эпидемиологические показатели.

```
function run_experiment(p)

    beta = p[:beta]

    β_und = fill(beta, 3)
    β_det = fill(beta / 10, 3)
```

```

model = initialize_sir(
    ;
    Ns = p[:Ns],
     $\beta_{und}$  =  $\beta_{und}$ ,
     $\beta_{det}$  =  $\beta_{det}$ ,
    infection_period = p[:infection_period],
    detection_time = p[:detection_time],
    death_rate = p[:death_rate],
    reinfection_probability = p[:reinfection_probability],
    Is = p[:Is],
    seed = p[:seed],
    n_steps = p[:n_steps],
)

infected_fraction(model) =
    count(a.status == :I for a in allagents(model)) / nagents(model)

peak_infected = 0.0

for step = 1:p[:n_steps]

    agent_ids = collect(allids(model))

    for id in agent_ids

```

```

    agent = try
      model[id]
    catch
      nothing
    end

    if agent != nothing
      sir_agent_step!(agent, model)
    end
  end

  frac = infected_fraction(model)

  if frac > peak_infected
    peak_infected = frac
  end
end

final_infected = infected_fraction(model)

final_recovered =
  count(a.status == :R for a in allagents(model)) / nagents(model)

total_deaths = sum(p[:Ns]) - nagents(model)

```

```

    return (
        peak = peak_infected,
        final_inf = final_infected,
        final_rec = final_recovered,
        deaths = total_deaths,
    )
end

```

run_experiment (generic function with 1 method)

4.3 Диапазон исследуемых параметров

Исследуем коэффициент заразности β в диапазоне от 0.1 до 1.0 с шагом 0.1.

```
beta_range = 0.1:0.1:1.0
```

```
0.1:0.1:1.0
```

Для каждого значения β выполняются три независимых прогона с различными значениями seed.

```
seeds = [42, 43, 44]
```

```
3-element Vector{Int64}:
```

```
42
```

```
43
```

```
44
```

Таким образом, всего будет выполнено:

10 значений $\beta \times 3$ значения seed = 30 экспериментов.

4.4 Формирование набора параметров

Создаём список словарей параметров для всех комбинаций коэффициента заразности и зерна генератора случайных чисел.

```
params_list = []

for b in beta_range
  for s in seeds
    push!(
      params_list,
      Dict{
        :beta => b,
        :Ns => [1000, 1000, 1000],
        :infection_period => 14,
        :detection_time => 7,
        :death_rate => 0.02,
        :reinfection_probability => 0.1,
        :Is => [0, 0, 1],
        :seed => s,
        :n_steps => 100,
      },
    )
  end
end
```

Проверим количество сформированных экспериментов.

```
length(params_list)
```

30

4.5 Выполнение серии экспериментов

Последовательно запускаем модель для каждого набора параметров. Полученные показатели объединяем с параметрами соответствующего эксперимента.

```
results = []

for params in params_list

  data = run_experiment(params)

  push!(
    results,
    merge(params, Dict(pairs(data))),
  )

  println(
    "Завершён эксперимент с beta = $(params[:beta]), " *
    "seed = $(params[:seed])",
  )
end
```

Завершён эксперимент с beta = 0.1, seed = 42

Завершён эксперимент с beta = 0.1, seed = 43

Завершён эксперимент с beta = 0.1, seed = 44

Завершён эксперимент с beta = 0.2, seed = 42

Завершён эксперимент с $\beta = 0.2$, seed = 43
Завершён эксперимент с $\beta = 0.2$, seed = 44
Завершён эксперимент с $\beta = 0.3$, seed = 42
Завершён эксперимент с $\beta = 0.3$, seed = 43
Завершён эксперимент с $\beta = 0.3$, seed = 44
Завершён эксперимент с $\beta = 0.4$, seed = 42
Завершён эксперимент с $\beta = 0.4$, seed = 43
Завершён эксперимент с $\beta = 0.4$, seed = 44
Завершён эксперимент с $\beta = 0.5$, seed = 42
Завершён эксперимент с $\beta = 0.5$, seed = 43
Завершён эксперимент с $\beta = 0.5$, seed = 44
Завершён эксперимент с $\beta = 0.6$, seed = 42
Завершён эксперимент с $\beta = 0.6$, seed = 43
Завершён эксперимент с $\beta = 0.6$, seed = 44
Завершён эксперимент с $\beta = 0.7$, seed = 42
Завершён эксперимент с $\beta = 0.7$, seed = 43
Завершён эксперимент с $\beta = 0.7$, seed = 44
Завершён эксперимент с $\beta = 0.8$, seed = 42
Завершён эксперимент с $\beta = 0.8$, seed = 43
Завершён эксперимент с $\beta = 0.8$, seed = 44
Завершён эксперимент с $\beta = 0.9$, seed = 42
Завершён эксперимент с $\beta = 0.9$, seed = 43
Завершён эксперимент с $\beta = 0.9$, seed = 44
Завершён эксперимент с $\beta = 1.0$, seed = 42
Завершён эксперимент с $\beta = 1.0$, seed = 43
Завершён эксперимент с $\beta = 1.0$, seed = 44

4.6 Формирование таблицы результатов

Результаты всех прогонов преобразуем в DataFrame. Каждая строка таблицы соответствует одному эксперименту.

```
df = DataFrame(results)
```

	n_steps	final_rec	Ns	death_rate	Is	beta	infection_period
	Int64	Float64	Array...	Float64	Array...	Float64	Int64
1	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	14
2	100	0.00266667	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	14
3	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	14
4	100	0.0400976	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	14
5	100	0.001	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	14
6	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	14
7	100	0.00748503	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	14
8	100	0.0606509	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	14
9	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	14
10	100	0.080938	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.4	14
11	100	0.0298954	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.4	14
12	100	0.000666667	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.4	14
13	100	0.00938086	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.5	14
14	100	0.000370233	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.5	14
15	100	0.000372995	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.5	14
16	100	0.0142643	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.6	14
17	100	0.00187829	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.6	14
18	100	0.00380228	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.6	14
19	100	0.0944487	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.7	14
20	100	0.0026296	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.7	14
21	100	0.0165414	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.7	14
22	100	0.134969	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.8	14
23	100	0.00224803	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.8	14
24	100	0.0237378	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.8	14
...	...	...	...	...	...	...	...

Выведем первые строки полученной таблицы.

```
first(df, 10)
```

	n_steps	final_rec	Ns	death_rate	Is	beta	infection_period
	Int64	Float64	Array...	Float64	Array...	Float64	Int64
1	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	14
2	100	0.00266667	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	14
3	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	14
4	100	0.0400976	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	14
5	100	0.001	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	14
6	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	14
7	100	0.00748503	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	14
8	100	0.0606509	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	14
9	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	14
10	100	0.080938	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.4	14

4.7 Сохранение результатов

Полные результаты всех прогонов сохраняются в CSV-файл. Это позволяет использовать их в дальнейшем анализе без повторного выполнения моделирования.

```
CSV.write(
    datadir("beta_scan_all.csv"),
    df,
)
```

```
"/home/alkamal/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--
study--simulation-modeling/labs/lab04/project/data/beta_scan_all.csv"
```

4.8 Усреднение результатов

Поскольку для каждого значения β выполнялось три независимых прогона, сгруппируем результаты по коэффициенту заразности.

Для каждого значения β вычисляем:

- среднюю пиковую долю инфицированных;
- среднюю конечную долю инфицированных;
- среднее количество умерших.

```
grouped = combine(  
    groupby(df, [:beta]),  
    :peak ⇒ mean ⇒ :mean_peak,  
    :final_inf ⇒ mean ⇒ :mean_final_inf,  
    :deaths ⇒ mean ⇒ :mean_deaths,  
)
```

	beta	mean_peak	mean_final_inf	mean_deaths
	Float64	Float64	Float64	Float64
1	0.1	0.000888889	0.0	0.0
2	0.2	0.332206	0.319967	44.0
3	0.3	0.666659	0.643955	208.0
4	0.4	0.666889	0.629722	226.667
5	0.5	1.0	0.996625	317.667
6	0.6	1.0	0.993352	348.0
7	0.7	1.0	0.962127	361.333
8	0.8	1.0	0.946348	356.333
9	0.9	1.0	0.963478	385.667
10	1.0	1.0	1.0	407.333

Выведем усреднённые результаты.

grouped

	beta	mean_peak	mean_final_inf	mean_deaths
	Float64	Float64	Float64	Float64
1	0.1	0.000888889	0.0	0.0
2	0.2	0.332206	0.319967	44.0
3	0.3	0.666659	0.643955	208.0
4	0.4	0.666889	0.629722	226.667
5	0.5	1.0	0.996625	317.667
6	0.6	1.0	0.993352	348.0
7	0.7	1.0	0.962127	361.333
8	0.8	1.0	0.946348	356.333
9	0.9	1.0	0.963478	385.667
10	1.0	1.0	1.0	407.333

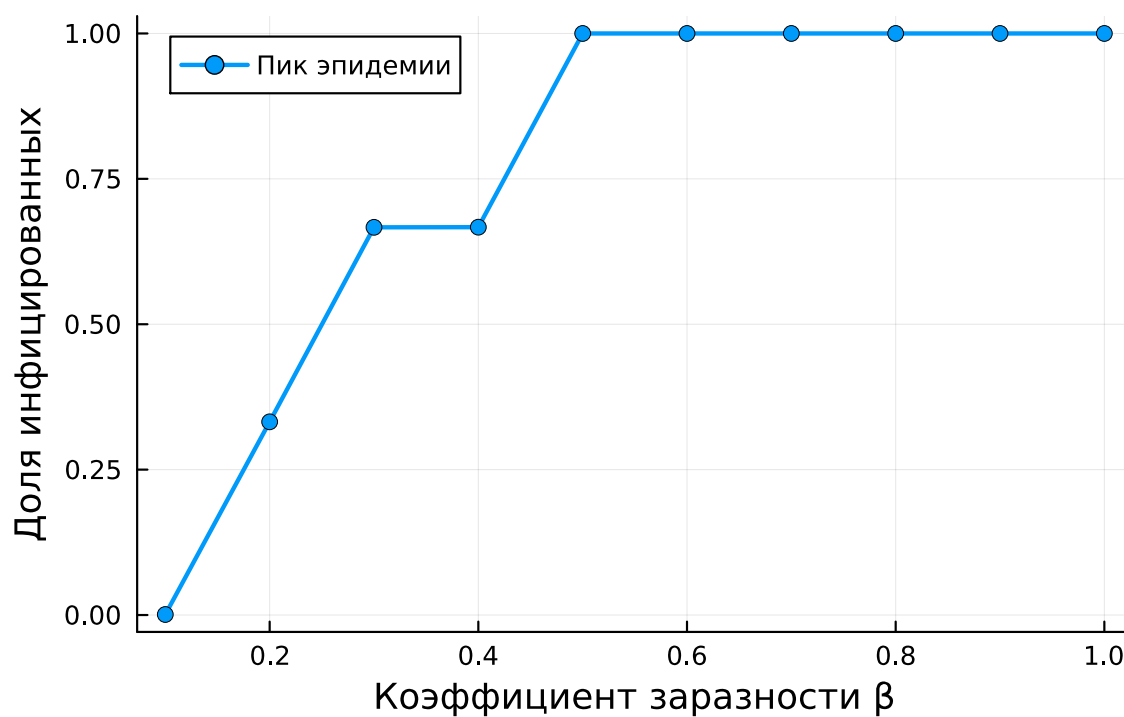
4.9 Визуализация результатов

Построим график зависимости основных эпидемиологических показателей от коэффициента заразности β .

Первая линия показывает среднее максимальное значение доли инфицированных агентов.

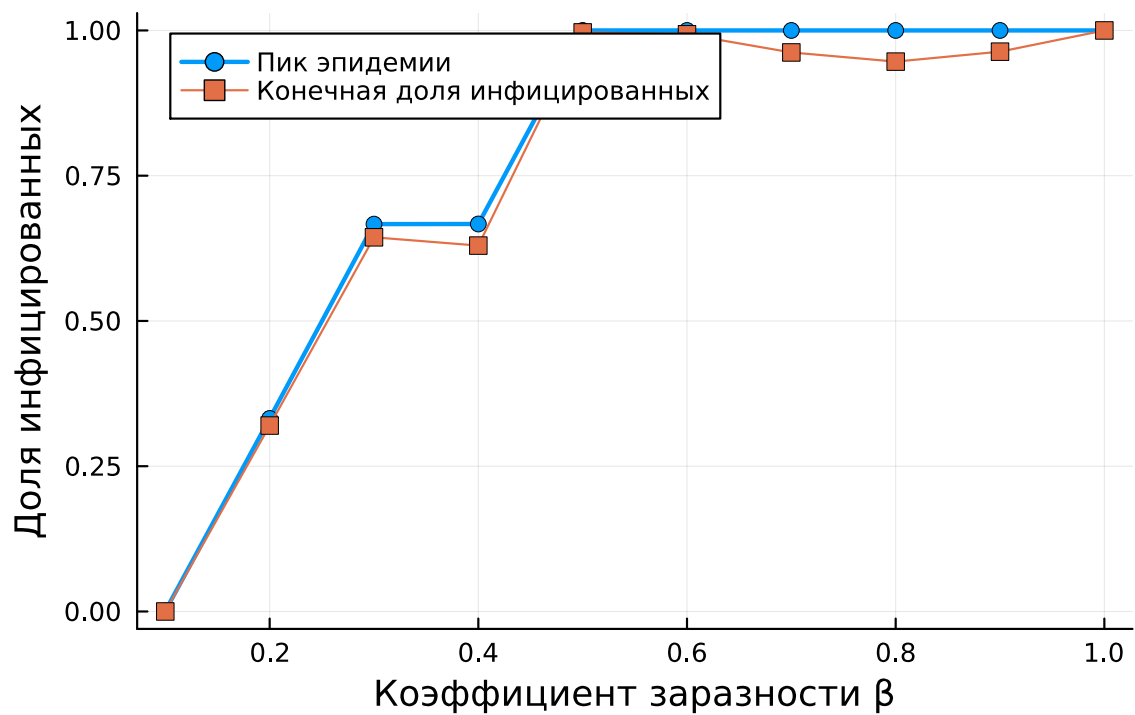
```
plot(  
    grouped.beta,  
    grouped.mean_peak,  
    label = "Пик эпидемии",  
    xlabel = "Коэффициент заразности  $\beta$ ",  
    ylabel = "Доля инфицированных",  
    marker = :circle,  
    linewidth = 2,
```

)



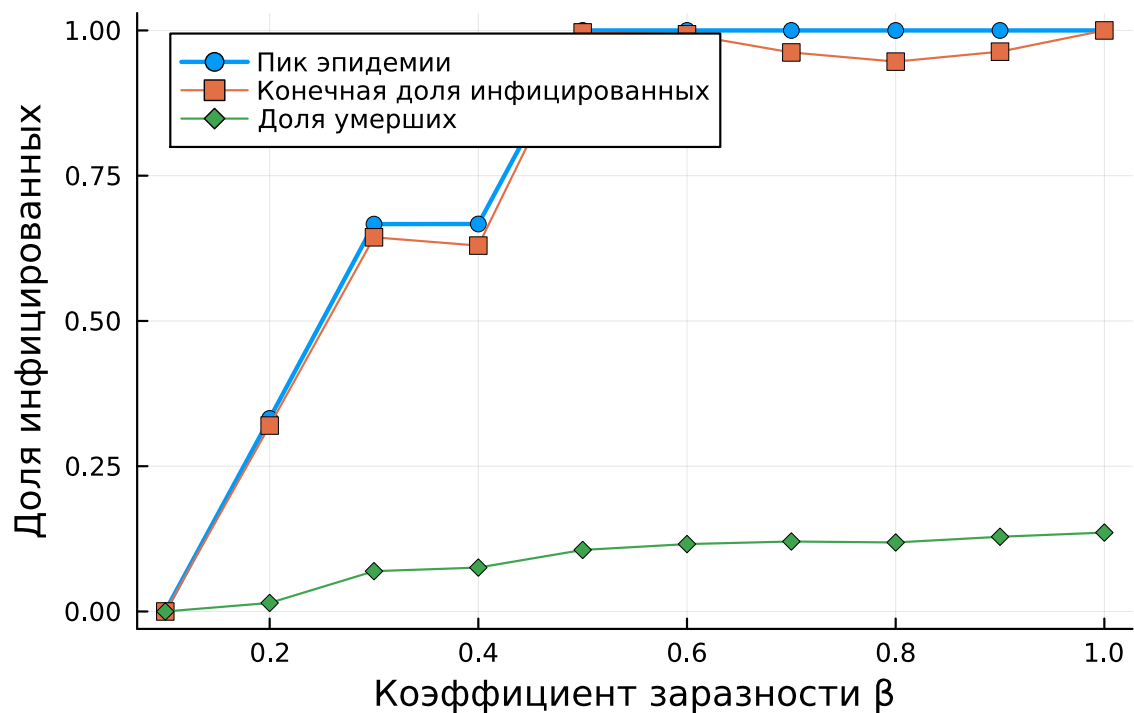
Добавим конечную долю инфицированных.

```
plot!(  
    grouped.beta,  
    grouped.mean_final_inf,  
    label = "Конечная доля инфицированных",  
    marker = :square,  
)
```



Количество умерших нормируем на начальную численность населения, равную 3000 агентам, и отображаем как долю.

```
plot!(
    grouped.beta,
    grouped.mean_deaths ./ 3000,
    label = "Доля умерших",
    marker = :diamond,
)
```



Сохраняем итоговый график в каталоге plots.

```
savefig(
    plotsdir("beta_scan.png"),
)
```

```
"/home/alkamal/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--
study--simulation-modeling/labs/lab04/project/plots/beta_scan.png"
```

4.10 Результаты эксперимента

В результате выполнения параметрического исследования создаются два основных файла:

- data/beta_scan_all.csv — результаты всех отдельных прогонов;
- plots/beta_scan.png — график зависимости эпидемиологических показателей от коэффициента заразности β .

```
println(  
    "Результаты сохранены в data/beta_scan_all.csv " *  
    "и plots/beta_scan.png",  
)
```

Результаты сохранены в data/beta_scan_all.csv и plots/beta_scan.png

4.11 Вывод

Проведён параметрический анализ агентной модели SIR при изменении коэффициента заразности β от 0.1 до 1.0. Для уменьшения влияния случайных флуктуаций каждое значение параметра исследовалось в трёх независимых прогонах.

Полученные результаты позволяют оценить влияние интенсивности передачи инфекции на максимальную долю инфицированных, конечное состояние эпидемии и смертность населения.

Построенный график позволяет сравнить эпидемиологические показатели при различных значениях β и выявить изменение характера распространения инфекции при росте заразности.

5 Исследование влияния интенсивности миграции

В данном эксперименте исследуется влияние миграции агентов между различными локациями на динамику распространения инфекции в агентной модели SIR.

Начальная популяция распределена между тремя локациями, при этом инфекция первоначально появляется только в первой локации. Интенсивность миграции изменяется от 0.0 до 0.5.

Для каждого значения интенсивности миграции выполняется несколько независимых прогонов модели с различными значениями seed.

В ходе эксперимента определяются два основных показателя:

- время достижения пика эпидемии (peak_time);
- максимальная доля инфицированных агентов (peak_value).

Полученные результаты позволяют оценить влияние перемещения агентов между локациями на скорость и интенсивность распространения инфекции.

5.1 Подключение библиотек

Активируем окружение проекта DrWatson и подключаем библиотеки, необходимые для агентного моделирования, обработки данных, статистического анализа, визуализации и сохранения результатов.

```
using DrWatson
@quickactivate "project"

using Agents
using DataFrames
using Plots
using CSV
using Random
using Statistics
```

Подключаем реализацию агентной модели SIR.

```
include(sourcedir("sir_model.jl"))
```

total_count (generic function with 1 method)

5.2 Формирование матрицы миграции

Для описания перемещения агентов между локациями определим функцию `create_migration_matrix`.

Параметр `C` задаёт количество локаций, а `intensity` — общую интенсивность миграции.

Диагональные элементы матрицы соответствуют вероятности остаться в текущей локации. Остальная вероятность равномерно распределяется между другими локациями.

```
function create_migration_matrix(C, intensity)

    M = ones(C, C) .* intensity ./ (C - 1)
```

```

    for i = 1:C
        M[i, i] = 1 - intensity
    end

    return M
end

```

`create_migration_matrix` (generic function with 1 method)

Например, при трёх локациях и интенсивности миграции 0.2 можно получить соответствующую матрицу переходов.

```
create_migration_matrix(3, 0.2)
```

3×3 Matrix{Float64}:

```

0.8  0.1  0.1
0.1  0.8  0.1
0.1  0.1  0.8

```

5.3 Определение времени достижения пика эпидемии

Создадим функцию `peak_time`, которая выполняет один эксперимент с заданными параметрами и определяет момент достижения максимальной доли инфицированных агентов.

```

function peak_time(p)

    migration_rates =
        create_migration_matrix(

```

```

        p[:C],
        p[:migration_intensity],
    )

model = initialize_sir(
    ;
    Ns = p[:Ns],
     $\beta_{und}$  = p[: $\beta_{und}$ ],
     $\beta_{det}$  = p[: $\beta_{det}$ ],
    infection_period = p[:infection_period],
    detection_time = p[:detection_time],
    death_rate = p[:death_rate],
    reinfection_probability = p[:reinfection_probability],
    Is = p[:Is],
    seed = p[:seed],
    migration_rates = migration_rates,
)

infected_frac(model) =
    count(a.status == :I for a in allagents(model)) / nagents(model)

peak = 0.0
peak_step = 0

for step = 1:p[:n_steps]

    agent_ids = collect(allids(model))

```

```

    for id in agent_ids

        agent = try
            model[id]
        catch
            nothing
        end

        if agent != nothing
            sir_agent_step!(agent, model)
        end
    end

    frac = infected_frac(model)

    if frac > peak
        peak = frac
        peak_step = step
    end
end

return (
    peak_time = peak_step,
    peak_value = peak,
)
end

```

peak_time (generic function with 1 method)

5.4 Диапазон интенсивности миграции

Исследуем значения интенсивности миграции от 0.0 до 0.5 с шагом 0.1.

```
migration_intensities = 0.0:0.1:0.5
```

0.0:0.1:0.5

Для каждого значения интенсивности выполняются три независимых прогона модели с различными значениями зерна генератора случайных чисел.

```
seeds = [42, 43, 44]
```

3-element Vector{Int64}:

42

43

44

Таким образом, всего выполняется:

6 значений интенсивности × 3 значения seed = 18 экспериментов.

5.5 Формирование набора параметров

Создаём список параметров для всех комбинаций интенсивности миграции и значений seed.

Начальная инфекция присутствует только в первой локации, что задаётся параметром Is = [1, 0, 0].

```

params_list = []

for mig in migration_intensities
  for s in seeds

    push!(
      params_list,
      Dict(
        :migration_intensity ⇒ mig,
        :C ⇒ 3,
        :Ns ⇒ [1000, 1000, 1000],
        :β_und ⇒ [0.5, 0.5, 0.5],
        :β_det ⇒ [0.05, 0.05, 0.05],
        :infection_period ⇒ 14,
        :detection_time ⇒ 7,
        :death_rate ⇒ 0.02,
        :reinfection_probability ⇒ 0.1,
        :Is ⇒ [1, 0, 0],
        :seed ⇒ s,
        :n_steps ⇒ 150,
      ),
    )
  end
end

```

Проверим количество сформированных экспериментов.

```
length(params_list)
```

18

5.6 Выполнение серии экспериментов

Для каждого набора параметров запускаем функцию `peak_time`. Полученные показатели объединяются с параметрами соответствующего эксперимента и сохраняются в общем массиве результатов.

```
results = []

for params in params_list

  data = peak_time(params)

  push!(
    results,
    merge(params, Dict{pairs(data)}),
  )

  println(
    "Завершён эксперимент с migration_intensity = " *
    "$(params[:migration_intensity]), seed = $(params[:seed])",
  )
end
```

Завершён эксперимент с migration_intensity = 0.0, seed = 42

Завершён эксперимент с migration_intensity = 0.0, seed = 43

Завершён эксперимент с migration_intensity = 0.0, seed = 44

Завершён эксперимент с migration_intensity = 0.1, seed = 42
Завершён эксперимент с migration_intensity = 0.1, seed = 43
Завершён эксперимент с migration_intensity = 0.1, seed = 44
Завершён эксперимент с migration_intensity = 0.2, seed = 42
Завершён эксперимент с migration_intensity = 0.2, seed = 43
Завершён эксперимент с migration_intensity = 0.2, seed = 44
Завершён эксперимент с migration_intensity = 0.3, seed = 42
Завершён эксперимент с migration_intensity = 0.3, seed = 43
Завершён эксперимент с migration_intensity = 0.3, seed = 44
Завершён эксперимент с migration_intensity = 0.4, seed = 42
Завершён эксперимент с migration_intensity = 0.4, seed = 43
Завершён эксперимент с migration_intensity = 0.4, seed = 44
Завершён эксперимент с migration_intensity = 0.5, seed = 42
Завершён эксперимент с migration_intensity = 0.5, seed = 43
Завершён эксперимент с migration_intensity = 0.5, seed = 44

5.7 Формирование таблицы результатов

Результаты всех отдельных прогонов преобразуем в DataFrame. Каждая строка таблицы соответствует одному запуску модели.

```
df = DataFrame(results)
```

	n_steps	Is	infection_period	detection_time	reinfection_probability	Ns
	Int64	Array...	Int64	Int64	Float64	Array...
1	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
2	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
3	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
4	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
5	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
6	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
7	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
8	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
9	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
10	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
11	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
12	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
13	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
14	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
15	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
16	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
17	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
18	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]

Выведем первые строки полученной таблицы.

```
first(df, 10)
```

	n_steps	Is	infection_period	detection_time	reinfection_probability	Ns
	Int64	Array...	Int64	Int64	Float64	Array...
1	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
2	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
3	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
4	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
5	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
6	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
7	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
8	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
9	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]
10	150	[1, 0, 0]	14	7	0.1	[1000, 1000, 1000]

5.8 Сохранение результатов

Полные результаты всех экспериментов сохраняем в CSV-файл для последующего анализа без необходимости повторного выполнения моделирования.

```
CSV.write(
  datadir("migration_scan_all.csv"),
  df,
)
```

```
"/home/alkamal/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--
study--simulation-modeling/labs/lab04/project/data/migration_scan_all.csv"
```

5.9 Усреднение результатов

Поскольку для каждого значения интенсивности миграции выполняется три независимых прогона, группируем результаты по параметру `migration_intensity`.

Для каждой группы рассчитываем:

- среднее время достижения пика эпидемии;
- среднюю максимальную долю инфицированных.

```
grouped = combine(  
    groupby(df, [:migration_intensity]),  
    :peak_time => mean => :mean_peak_time,  
    :peak_value => mean => :mean_peak_value,  
)
```

	migration_intensity	mean_peak_time	mean_peak_value
	Float64	Float64	Float64
1	0.0	15.0	0.333259
2	0.1	22.0	1.0
3	0.2	16.0	1.0
4	0.3	17.0	1.0
5	0.4	19.0	1.0
6	0.5	20.0	1.0

Выведем таблицу усреднённых результатов.

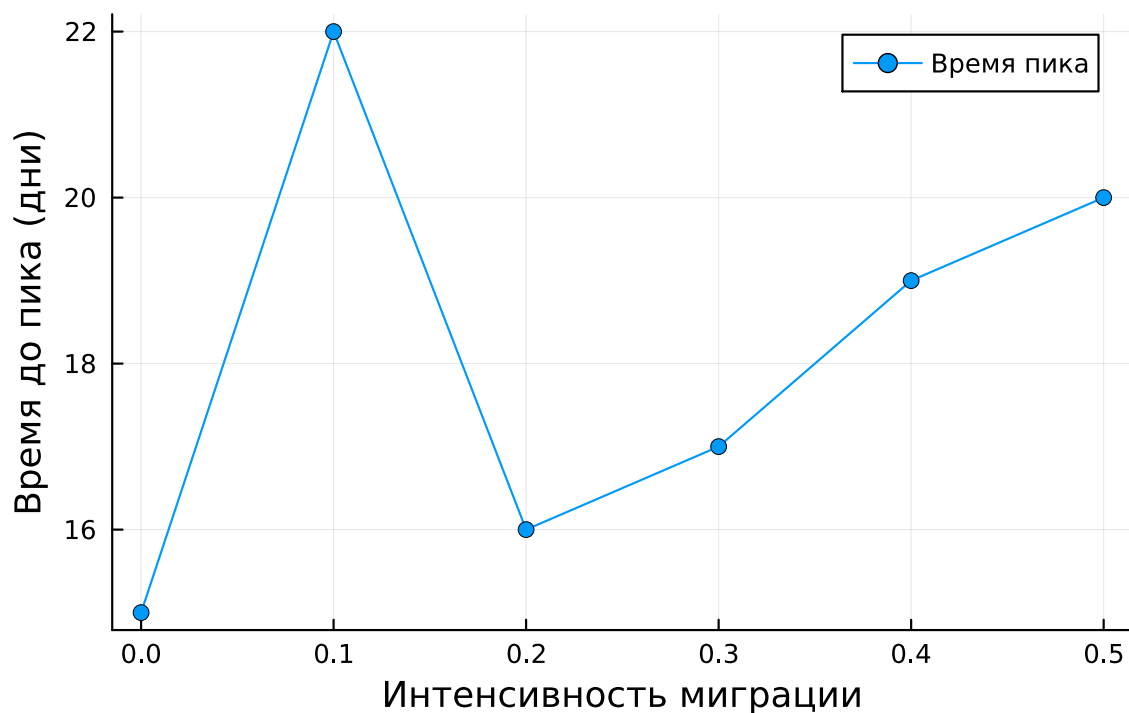
```
grouped
```

	migration_intensity	mean_peak_time	mean_peak_value
	Float64	Float64	Float64
1	0.0	15.0	0.333259
2	0.1	22.0	1.0
3	0.2	16.0	1.0
4	0.3	17.0	1.0
5	0.4	19.0	1.0
6	0.5	20.0	1.0

5.10 Визуализация результатов

Построим график зависимости времени достижения пика эпидемии от интенсивности миграции.

```
plot(  
    grouped.migration_intensity,  
    grouped.mean_peak_time,  
    marker = :circle,  
    xlabel = "Интенсивность миграции",  
    ylabel = "Время до пика (дни)",  
    label = "Время пика",  
)
```

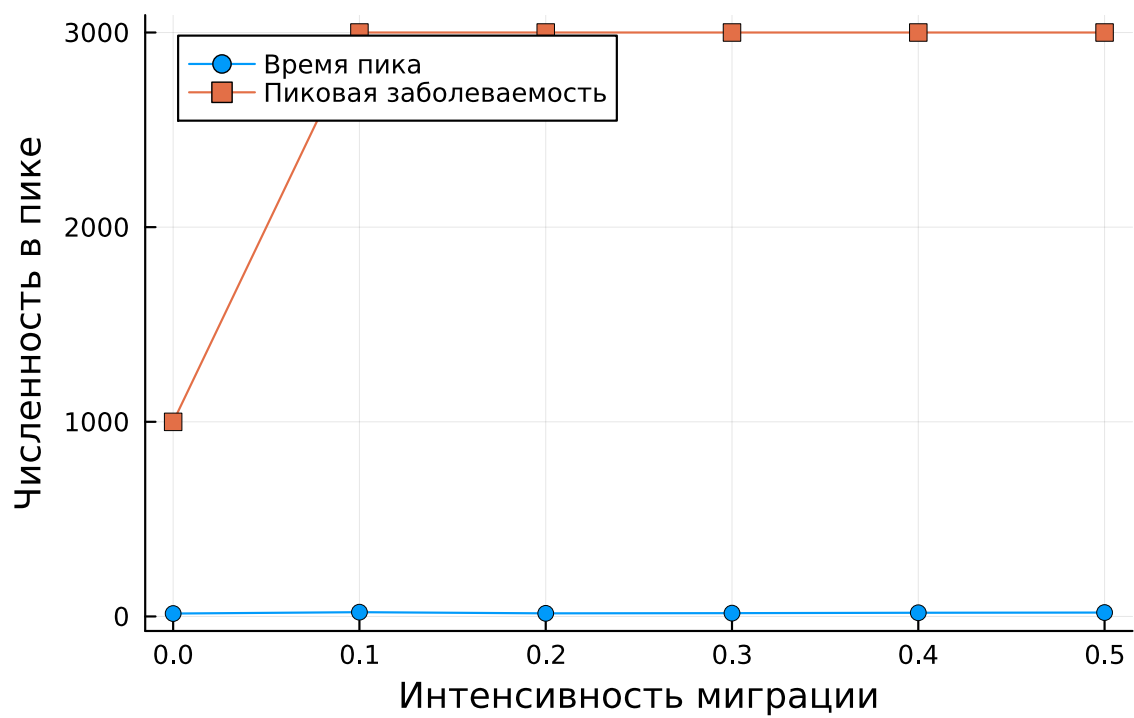


Добавим на график пиковую заболеваемость.

Значение `mean_peak_value` представляет долю инфицированных. Для отображения приблизительной численности инфицированных эта величина

умножается на начальную численность популяции, равную 3000 агентов.

```
plot!(
    grouped.migration_intensity,
    grouped.mean_peak_value .* 3000,
    marker = :square,
    xlabel = "Интенсивность миграции",
    ylabel = "Численность в пике",
    label = "Пиковая заболеваемость",
)
```



Сохраняем итоговый график в каталоге plots.

```
savefig(
    plotsdir("migration_effect.png"),
)
```

```
"/home/alkamal/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--  
study--simulation-modeling/labs/lab04/project/plots/migration_effect.png"
```

5.11 Результаты эксперимента

После выполнения эксперимента создаются следующие файлы:

- data/migration_scan_all.csv — результаты всех отдельных прогонов модели;
- plots/migration_effect.png — график зависимости показателей эпидемии от интенсивности миграции.

```
println(  
    "Результаты сохранены в data/migration_scan_all.csv " *  
    "и plots/migration_effect.png",  
)
```

Результаты сохранены в data/migration_scan_all.csv и plots/migration_effect.png

5.12 Вывод

Проведено исследование влияния интенсивности миграции агентов между тремя локациями на динамику распространения инфекции. Интенсивность миграции изменялась от 0.0 до 0.5, а для каждого значения выполнялось три независимых прогона модели.

В качестве основных характеристик использовались время достижения пика эпидемии и максимальная доля инфицированных агентов. Полученные результаты позволяют сравнить скорость и интенсивность развития эпидемического процесса при различных уровнях миграции населения между локациями.

6 Многокритериальная оптимизация параметров модели SIR

В данном эксперименте выполняется многокритериальная оптимизация параметров агентной модели SIR.

Цель оптимизации состоит в одновременном уменьшении двух показателей:

- максимальной доли инфицированных агентов;
- доли умерших агентов.

В качестве изменяемых параметров рассматриваются:

- коэффициент передачи инфекции β_{und} ;
- время выявления инфекции `detection_time`;
- вероятность смерти `death_rate`.

Для поиска компромиссных решений используется многокритериальный эволюционный алгоритм Borg MOEA из пакета `BlackBoxOptim`.

6.1 Подключение библиотек

Активируем окружение проекта `DrWatson` и подключаем библиотеки, необходимые для оптимизации, статистической обработки результатов и работы с агентной моделью.

```
using DrWatson
@quickactivate "project"

using BlackBoxOptim
using Random
using Statistics
using Agents
using JLD2
```

Подключаем реализацию агентной модели SIR.

```
include(sourcedir("sir_model.jl"))
```

total_count (generic function with 1 method)

6.2 Целевая функция многокритериальной оптимизации

Определим функцию `cost_multi`, которая принимает вектор оптимизируемых параметров x .

Компоненты вектора интерпретируются следующим образом:

- $x[1]$ — коэффициент передачи инфекции β_{und} ;
- $x[2]$ — время выявления инфекции `detection_time`;
- $x[3]$ — вероятность смерти `death_rate`.

Функция возвращает два значения:

1. среднюю пиковую долю инфицированных;
2. среднюю долю умерших.

Оба показателя должны быть минимизированы.

```
function cost_multi(x)

    β_und = fill(x[1], 3)
    β_det = fill(x[1] / 10, 3)

    detection_time = round(Int, x[2])

    death_rate = x[3]

    infected_frac(model) =
        count(a.status == :I for a in allagents(model)) / nagents(model)

    dead_count(model) =
        3000 - nagents(model)

    replicates = 5
```

```

peak_vals = Float64[]
dead_vals = Int[]

for rep = 1:replicates

    model = initialize_sir(
        ;
        Ns = [1000, 1000, 1000],
         $\beta_{und}$  =  $\beta_{und}$ ,
         $\beta_{det}$  =  $\beta_{det}$ ,
        infection_period = 14,
        detection_time = detection_time,
        death_rate = death_rate,
        reinfection_probability = 0.1,
        Is = [0, 0, 1],
        seed = 42 + rep,
        n_steps = 100,
    )

    peak_infected = 0.0

```

```

    for step = 1:100

        Agents.step!(model, 1)

        frac = infected_frac(model)

        if frac > peak_infected
            peak_infected = frac
        end
    end

    push!(peak_vals, peak_infected)
    push!(dead_vals, dead_count(model))
end

mean_peak = mean(peak_vals)
mean_death_fraction = mean(dead_vals) / 3000

return (
    mean_peak,
    mean_death_fraction,
)
end

```

`cost_multi` (generic function with 1 method)

6.3 Диапазоны оптимизируемых параметров

Оптимизация выполняется в следующих пределах:

- β_{und} от 0.1 до 1.0;
- `detection_time` от 3 до 14 дней;
- `death_rate` от 0.01 до 0.1.

```
search_range = [  
    (0.1, 1.0),  
    (3.0, 14.0),  
    (0.01, 0.1),  
]
```

3-element Vector{Tuple{Float64, Float64}}:

```
(0.1, 1.0)  
(3.0, 14.0)  
(0.01, 0.1)
```

6.4 Запуск многокритериальной оптимизации

Используем алгоритм `borg_moea`.

Поскольку одновременно оптимизируются два показателя, задаётся схема `ParetoFitnessScheme{2}`.

Значение `is_minimizing = true` указывает, что оба критерия необходимо минимизировать.

Максимальное время работы алгоритма составляет 120 секунд.

```

result = bboptimize(
    cost_multi,
    Method = :borg_moea,
    FitnessScheme = ParetoFitnessScheme{2}(
        is_minimizing = true,
    ),
    SearchRange = search_range,
    NumDimensions = 3,
    MaxTime = 120,
    TraceMode = :compact,
)

```

Starting optimization with optimizer BlackBoxOptim.BorgMOEA{BlackBoxOptim.EpsBoxDominanceFit
0.00 secs, 0 evals, 0 steps
pop.size=50 arch.size=0 n.restarts=0

Optimization stopped after 1 steps and 284.59 seconds
Termination reason: Max time (120.0 s) reached
Steps per second = 0.00
Function evals per second = 0.18
Improvements/step = NaN
Total function evaluations = 51

Best candidate found: [0.111217, 4.27382, 0.0768941]

Fitness: (0.00040, 0.00000) agg=0.00040

BlackBoxOptim.OptimizationResults("borg_moea", "Max time (120.0 s) reached", 1, 1.7884838229

```
1.0, :TargetFitness => nothing, :TraceMode => :compact, :FitnessScheme => ScalarFitnessScheme,
50, :NumDimensions => :NotSpecified, :FitnessTolerance => 1.0e-8, :TraceInterval => 0.5, :Max
```

6.5 Получение результата оптимизации

Из результата оптимизации извлекаем найденный набор параметров и соответствующие значения целевых функций.

```
best = best_candidate(result)
fitness = best_fitness(result)
```

```
(0.0004, 0.0)
```

6.6 Вывод оптимальных параметров

Выведем найденные значения параметров модели.

```
println("Оптимальные параметры:")
println("β_und = ${best[1]}")
println(
    "Время выявления = ${round(Int, best[2])} дней",
)
println("Смертность = ${best[3]}")
```

Оптимальные параметры:

β_und = 0.11121687478172426

Время выявления = 4 дней

Смертность = 0.07689412214024807

Выведем достигнутые значения критериев.

```
println("Достигнутые показатели:")
println("Пик заболеваемости: ${fitness[1]}")
println("Доля умерших: ${fitness[2]}")
```

Достигнутые показатели:

Пик заболеваемости: 0.0004

Доля умерших: 0.0

6.7 Сохранение результатов

Результат оптимизации сохраняется в файл JLD2.

В файл записываются:

- best — найденный набор параметров;
- fitness — соответствующие значения целевых функций.

```
save(
    datadir("optimization_result.jld2"),
    Dict(
        "best" => best,
        "fitness" => fitness,
    ),
)
```

6.8 Вывод

В результате выполнена многокритериальная оптимизация параметров агентной модели SIR. Одновременно минимизировались пиковая заболеваемость и доля умерших.

Использование нескольких независимых прогонов для каждого набора параметров позволяет уменьшить влияние случайности, характерной для агентного моделирования.

Алгоритм Borg MOEA позволяет искать компромиссные решения между двумя конкурирующими критериями и определить набор параметров, обеспечивающий более благоприятную динамику эпидемического процесса.

7 Комплексный анализ результатов сканирования β

В данном разделе выполняется итоговый анализ результатов параметрического исследования коэффициента заразности β .

Результаты моделирования предварительно были сохранены в файл `beta_scan_all.csv`.

На основе сохранённых данных строятся три графика:

- пиковая и конечная доля инфицированных;
- число умерших;
- конечная доля выздоровевших.

Это позволяет комплексно оценить влияние коэффициента заразности β на основные характеристики эпидемического процесса.

7.1 Подключение библиотек

Активируем окружение проекта DrWatson и подключаем библиотеки, необходимые для работы с табличными данными, загрузки CSV-файлов и визуализации результатов.

```
using DrWatson
@quickactivate "project"

using Agents
using DataFrames
using Plots
using CSV
```

Подключаем реализацию агентной модели SIR.

В данном скрипте модель повторно не запускается, однако файл подключается для сохранения общей структуры проекта.

```
include(scrdir("sir_model.jl"))
```

total_count (generic function with 1 method)

7.2 Загрузка результатов экспериментов

Загружаем результаты сканирования коэффициента β , ранее сохранённые в файл data/beta_scan_all.csv.

```
df = CSV.read(
    datadir("beta_scan_all.csv"),
    DataFrame,
)
```

	n_steps	final_rec	Ns	death_rate	Is	beta	infection_period
	Int64	Float64	String31	Float64	String15	Float64	Int64
1	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	14
2	100	0.00266667	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	14
3	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	14
4	100	0.0400976	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	14
5	100	0.001	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	14
6	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	14
7	100	0.00748503	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	14
8	100	0.0606509	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	14
9	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	14
10	100	0.080938	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.4	14
11	100	0.0298954	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.4	14
12	100	0.000666667	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.4	14
13	100	0.00938086	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.5	14
14	100	0.000370233	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.5	14
15	100	0.000372995	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.5	14
16	100	0.0142643	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.6	14
17	100	0.00187829	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.6	14
18	100	0.00380228	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.6	14
19	100	0.0944487	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.7	14
20	100	0.0026296	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.7	14
21	100	0.0165414	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.7	14
22	100	0.134969	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.8	14
23	100	0.00224803	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.8	14
24	100	0.0237378	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.8	14
...	...	...	...	...	...	...	...

Выведем первые строки таблицы для проверки структуры данных.

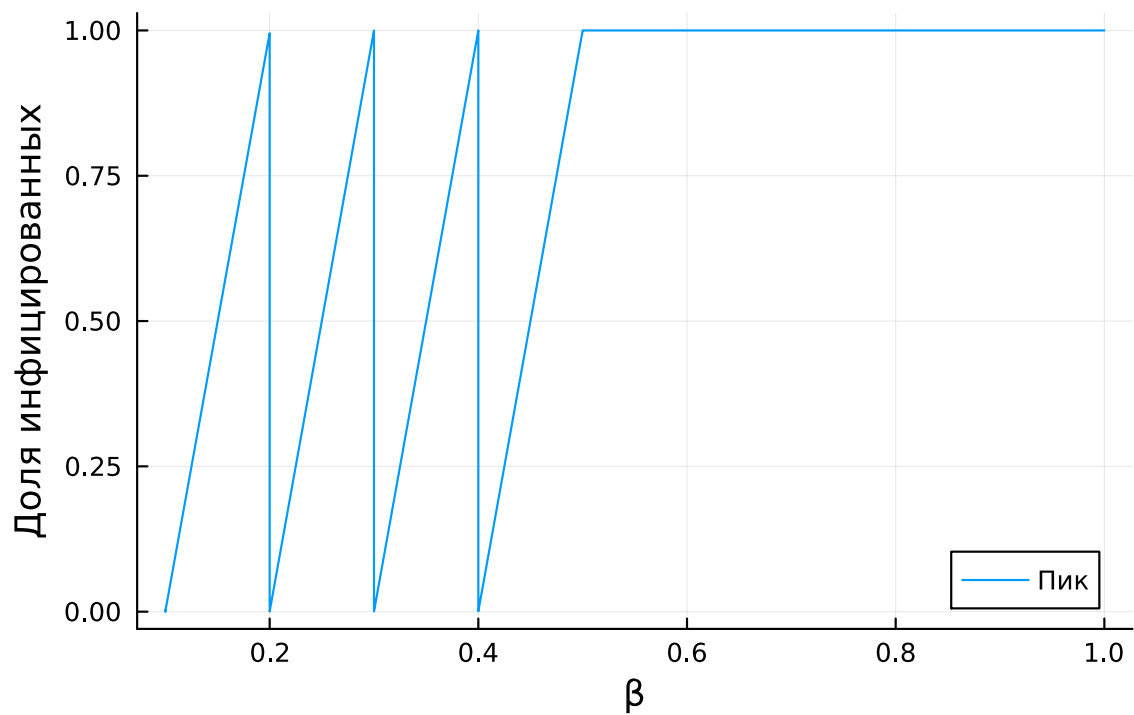
```
first(df, 10)
```

	n_steps	final_rec	Ns	death_rate	Is	beta	infection_period
	Int64	Float64	String31	Float64	String15	Float64	Int64
1	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	14
2	100	0.00266667	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	14
3	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	14
4	100	0.0400976	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	14
5	100	0.001	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	14
6	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	14
7	100	0.00748503	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	14
8	100	0.0606509	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	14
9	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	14
10	100	0.080938	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.4	14

7.3 Анализ доли инфицированных

Первый график показывает зависимость пикового значения доли инфицированных от коэффициента заразности β .

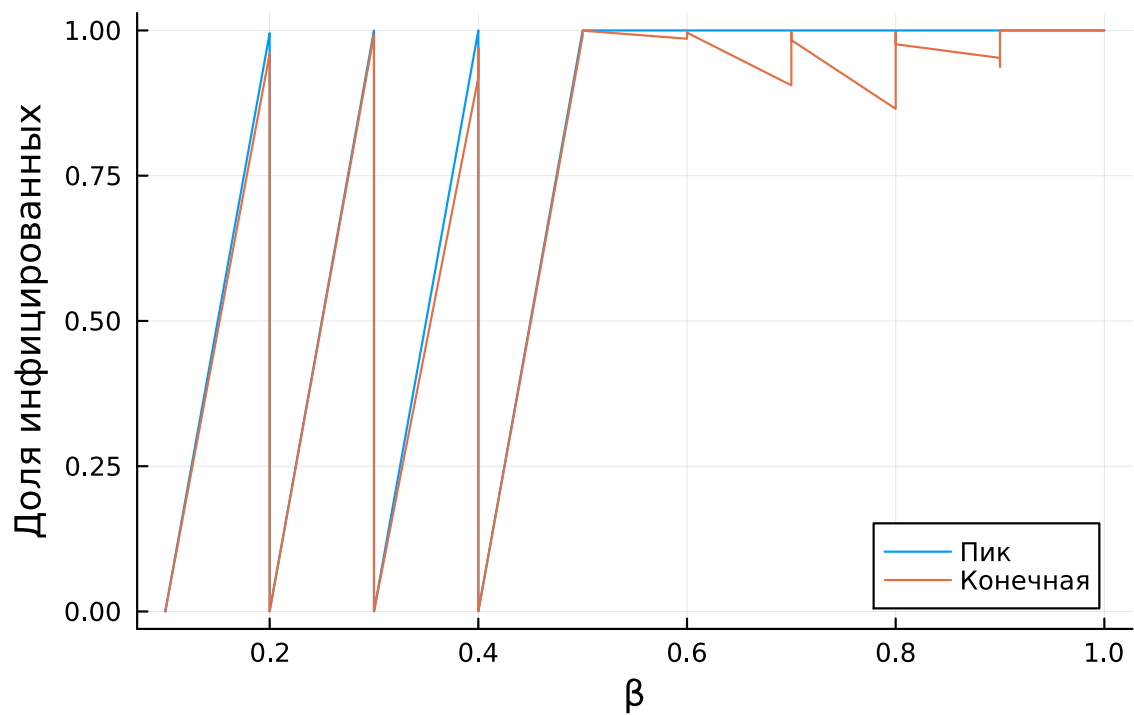
```
p1 = plot(
    df.beta,
    df.peak,
    label = "Пик",
    xlabel = " $\beta$ ",
    ylabel = "Доля инфицированных",
)
```



На тот же график добавляем конечную долю инфицированных.

Это позволяет сравнить максимальный уровень распространения инфекции с состоянием системы в конце моделирования.

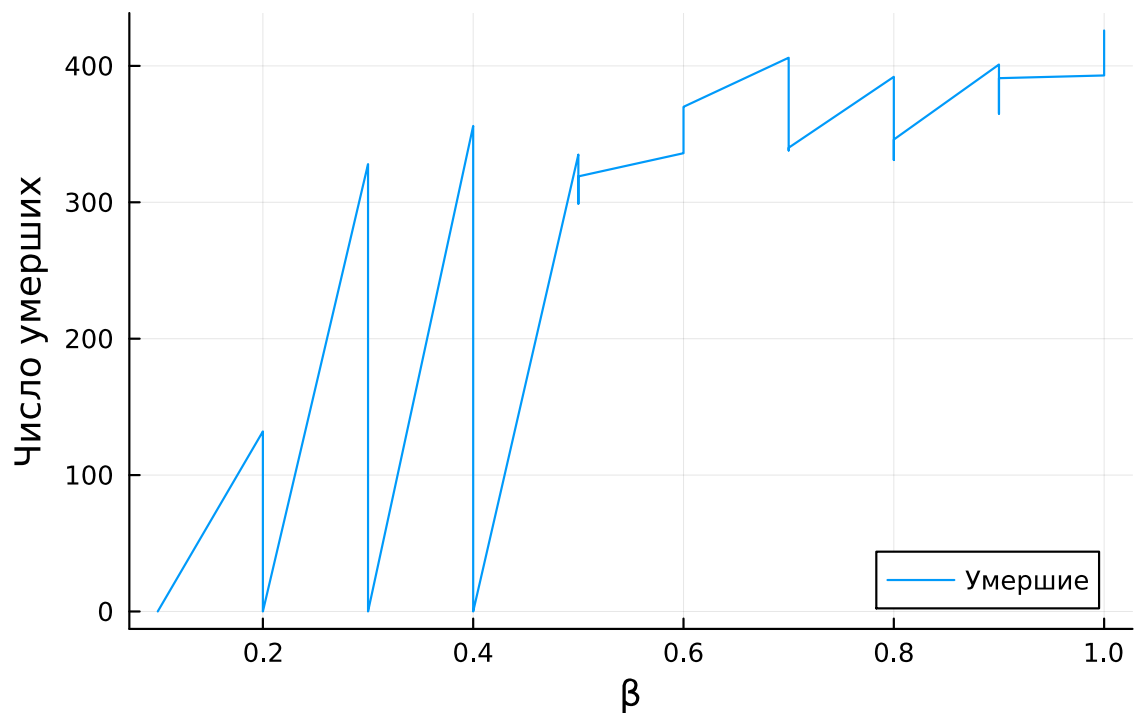
```
plot!(
    p1,
    df.beta,
    df.final_inf,
    label = "Конечная",
)
```



7.4 Анализ смертности

Второй график отражает зависимость общего числа умерших от значения коэффициента заразности β .

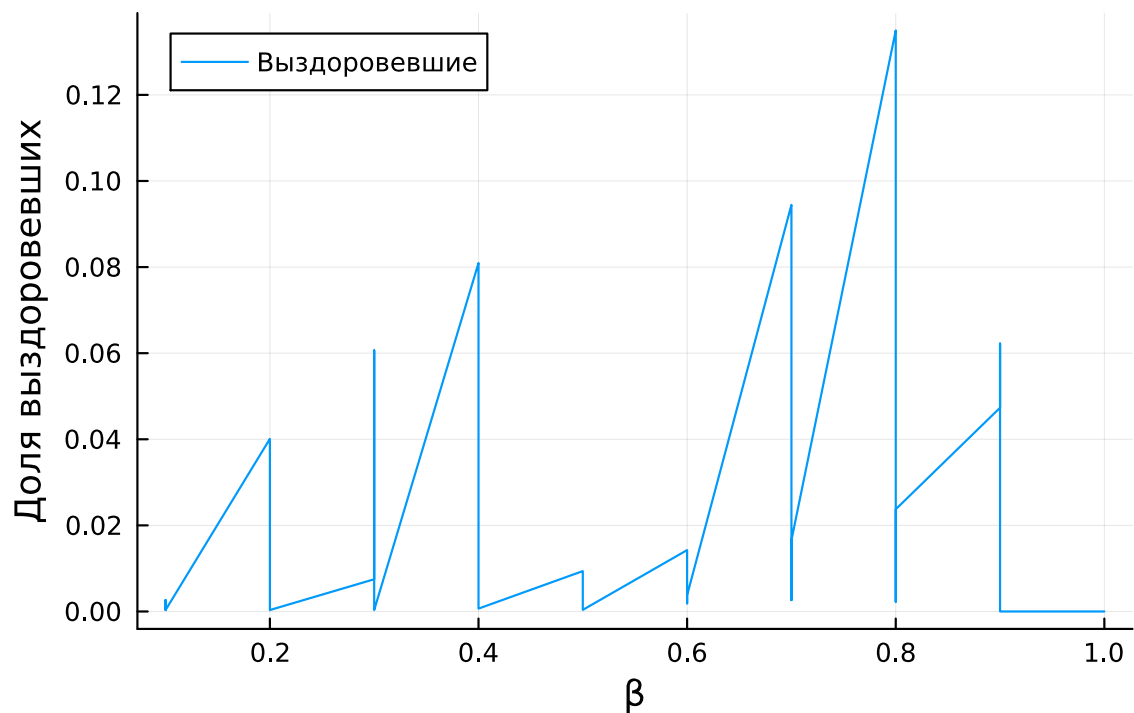
```
p2 = plot(
  df.beta,
  df.deaths,
  xlabel = "β",
  ylabel = "Число умерших",
  label = "Умершие",
)
```



7.5 Анализ выздоровления

Третий график показывает конечную долю выздоровевших агентов при различных значениях коэффициента заразности.

```
p3 = plot(
    df.beta,
    df.final_rec,
    xlabel = "β",
    ylabel = "Доля выздоровевших",
    label = "Выздоровевшие",
)
```

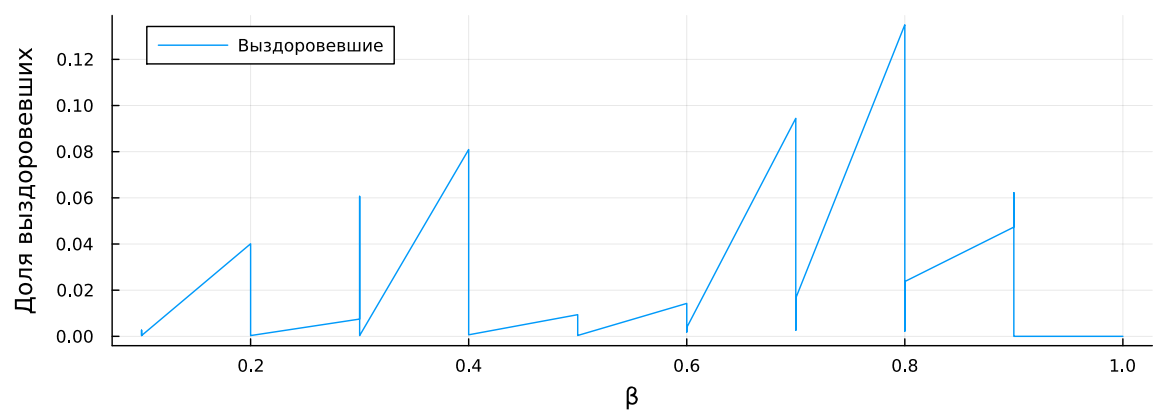
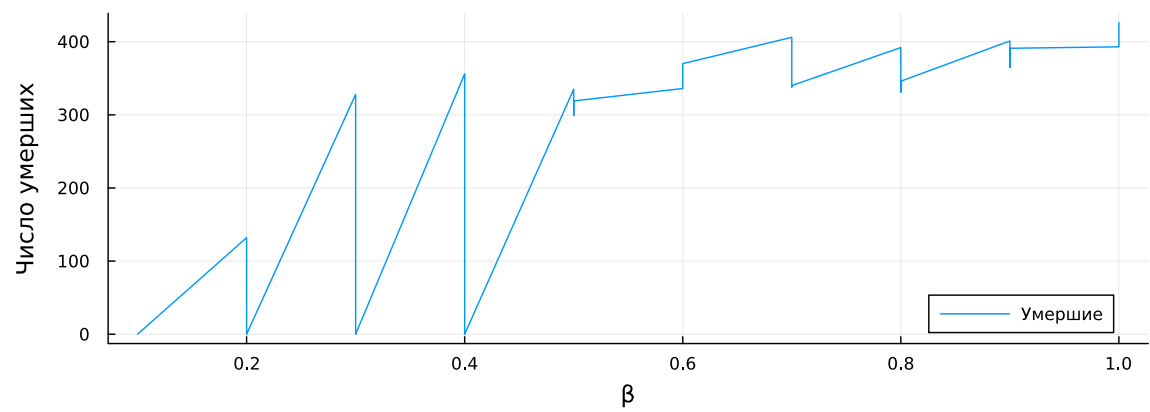
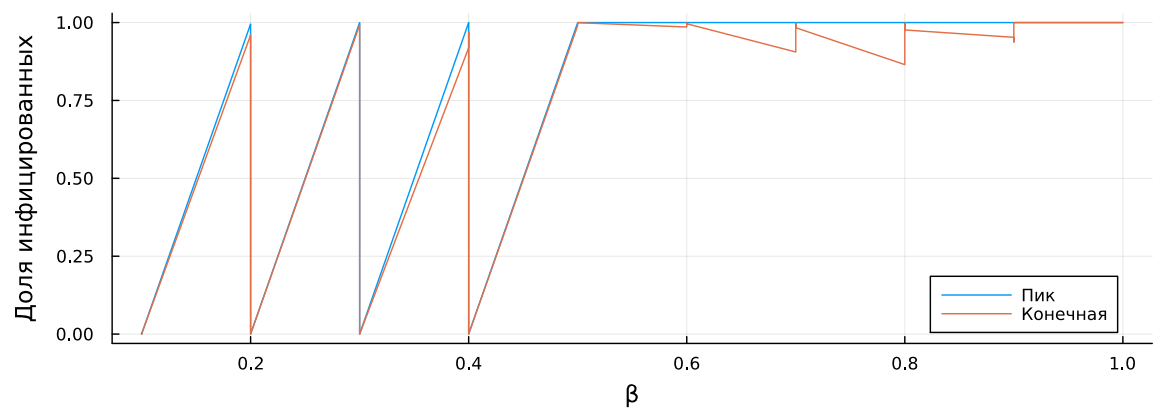


7.6 Формирование составного графика

Объединяем три полученных графика в одну вертикальную композицию.

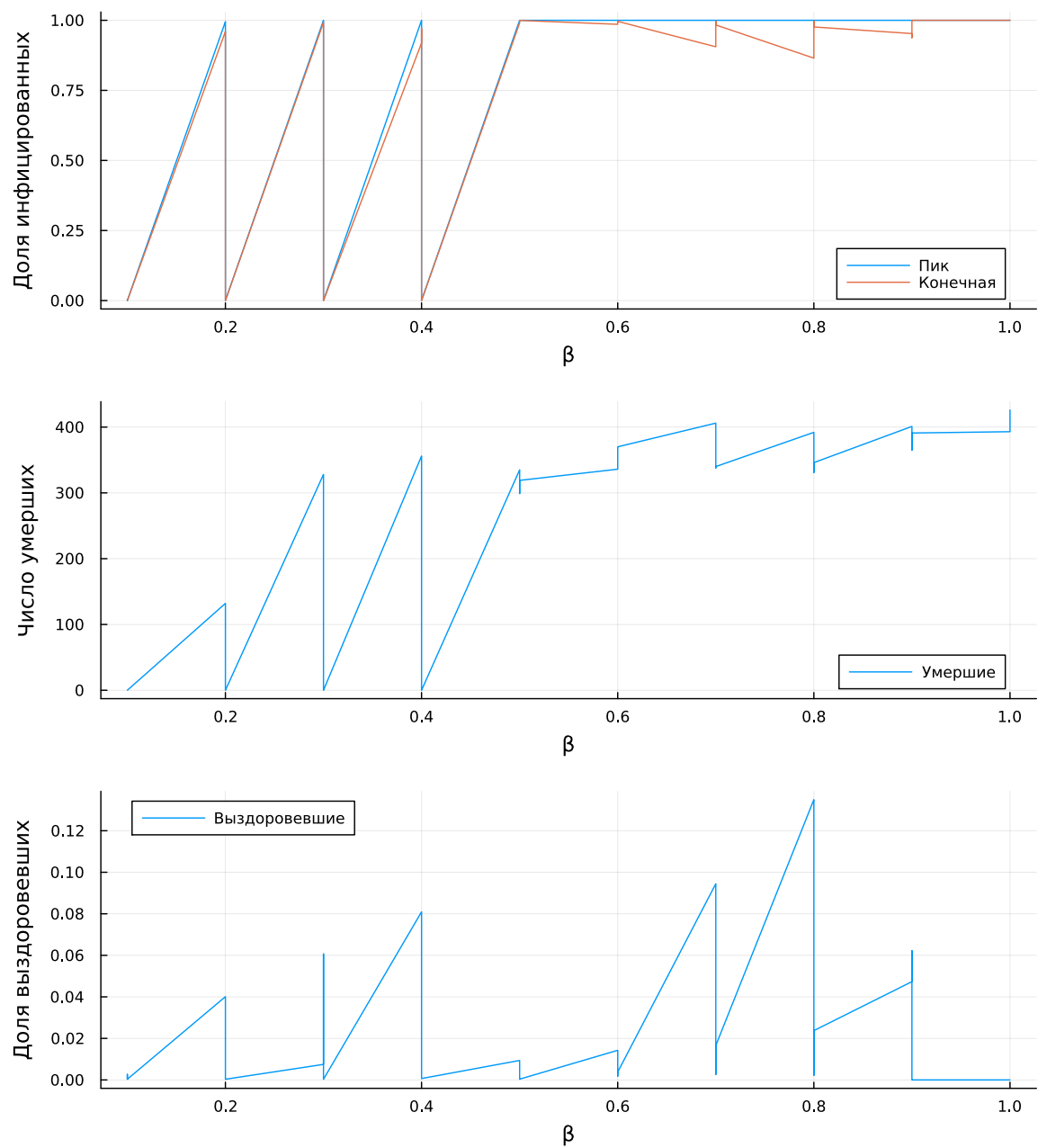
Такая форма представления позволяет одновременно сравнить изменение основных эпидемиологических показателей при росте β .

```
combined_plot = plot(
    p1,
    p2,
    p3,
    layout = (3, 1),
    size = (800, 900),
)
```



Отображаем составной график.

```
combined_plot
```



7.7 Сохранение результата

Сохраняем итоговую визуализацию в каталог plots.

```
savefig(  
    combined_plot,  
    plotsdir("comprehensive_analysis.png"),  
)
```

```
"/home/alkamal/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--  
study--simulation-modeling/labs/lab04/project/plots/comprehensive_analysis.png"
```

7.8 Результат анализа

В результате формируется файл:

plots/comprehensive_analysis.png

Он содержит три взаимосвязанных графика, отражающих влияние коэффициента заразности на пиковую и конечную заболеваемость, смертность и долю выздоровевших агентов.

```
println(  
    "Комплексный анализ сохранён в " *  
    "plots/comprehensive_analysis.png",  
)
```

Комплексный анализ сохранён в plots/comprehensive_analysis.png

7.9 Вывод

Проведён комплексный анализ результатов параметрического исследования коэффициента заразности β .

Совместное рассмотрение пикового уровня инфекции, конечной доли инфицированных, числа умерших и доли выздоровевших позволяет оценить

влияние изменения интенсивности передачи инфекции на динамику агентной модели SIR.

Полученная визуализация объединяет основные показатели эксперимента и может использоваться для итогового сравнения результатов в отчёте по лабораторной работе.

8 Выводы

В ходе лабораторной работы была реализована и исследована агентная модель распространения эпидемии SIR. Проведено базовое моделирование динамики восприимчивых, инфицированных и выздоровевших агентов. Исследовано влияние коэффициента заразности β и интенсивности миграции на показатели эпидемии. Выполнена многокритериальная оптимизация параметров модели и проведена визуализация полученных результатов. Результаты экспериментов сохранены в виде графиков и файлов данных, а литературные версии программ представлены в формате Quarto и Jupyter Notebook.